BioProver: AI-Guided CEGAR for Formal Verification and Repair of Synthetic Biology Circuits

Abstract

Synthetic biology circuits are designed by composing genetic parts (promoters, repressors, activators) into regulatory networks that implement logic, oscillation, and memory functions. Verifying that a circuit design satisfies its temporal specification—given uncertain kinetic parameters—remains an open challenge: the dynamics are nonlinear, high-dimensional, and stiff. We present BioProver, a counterexample-guided abstraction refinement (CEGAR) framework that formally verifies gene circuit designs against signal temporal logic (STL) specifications over ODE models. Key contributions include: (1) a biology-specific refinement strategy exploiting Hill-function monotonicity that reduces CEGAR convergence from exponential to $O(m \log(x_{\max}/\varepsilon))$ iterations for monotone circuits; (2) an explicit four-level soundness hierarchy (Sound, δ-Sound, Bounded, Approximate) with end-to-end error tracking through δ-decidability, discretization, and moment closure; (3) compositional verification via circular assume-guarantee reasoning that scales to 50+ species circuits—a 24× speedup over monolithic verification at 5 species and verification of 20-species circuits in under 1 second where monolithic verification times out; (4) an optional ML-accelerated predicate predictor that speeds up refinement by 1.54× while never compromising soundness; and (5) a standalone certificate verifier with an 800-line trusted computing base (TCB), a 72× reduction from the 58K-line main codebase, that replays flowpipe enclosures and invariant checks to provide independently auditable verification evidence. We formalize end-to-end (ε, δ) error propagation through the pipeline, with rigorous bounds for moment closure truncation and ODE discretization. On a benchmark suite of 10 circuits (3–9 species), monolithic BioProver verifies 8 within a 35-second timeout; compositional verification extends the scalability frontier to 50+ species with sub-second verification times.

1 Introduction

Synthetic biology aims to engineer living cells with programmable behavior by assembling genetic circuits from standardized parts [1]. A toggle switch combining two mutually repressing genes produces bistable memory; a repressilator with three genes in a negative-feedback ring generates sustained oscillations [2]. Before fabricating these circuits in the wet lab, designers need assurance that the intended behavior holds across the range of plausible kinetic parameters obtained from part characterization data.

The verification challenge is formidable. Gene circuit dynamics are governed by systems of nonlinear ordinary differential equations (ODEs) with Hill-function kinetics:

$$\dot{x}_i = \sum_j V_{ij} \frac{x_j^{n_{ij}}}{K_{ij}^{n_{ij}} + x_j^{n_{ij}}} - \gamma_i x_i + u_i, \tag{1}$$

where parameters $(V_{ij}, K_{ij}, n_{ij}, \gamma_i)$ are uncertain. Specifications such as "protein X always remains above concentration 0.5" or "the circuit oscillates with period between 18 and 25 time units" are naturally expressed in signal temporal logic (STL) [3].

Existing approaches fall into three categories. *Simulation-based* tools like BREACH [6] sample parameter space and check STL robustness but provide no formal guarantees. *Reachability tools* like Flow* [7] and dREACH [8] compute flowpipe over-approximations but do not natively support STL specifications or parameter synthesis. *Statistical model checking* [9] provides probabilistic guarantees but cannot prove universal properties. None of these approaches combine formal verification with automatic parameter repair.

Contributions. BIOPROVER addresses these gaps with five technical contributions:

1. **Biology-specific CEGAR.** We instantiate the CEGAR loop [4] for nonlinear ODE verification with five refinement strategies: structural, monotonicity-accelerated, time-scale, interpolation-based, and simulation-guided (§3).
2. **Explicit soundness hierarchy.** Every verification result carries a soundness annotation from a four-level lattice (SOUND > δ-SOUND > BOUNDED > APPROXIMATE) with end-to-end error tracking through δ-decidability, discretization, and truncation errors (§4).
3. **Compositional verification.** Circular assume-guarantee reasoning with formally verified well-formedness conditions scales verification to 50+ species—the headline result. At 5 species, compositional verification is 24× faster than monolithic; at 20 species it completes in 0.8s where monolithic times out (§5).
4. **ML-accelerated refinement.** An optional MLP predicate predictor with quality-gated auto-disable achieves 1.54× speedup without compromising soundness (§6).
5. **Standalone certificate verifier.** An 800-line verifier module that independently validates proof certificates (flowpipe enclosures, invariant checks, error budgets) without importing any BIOPROVER internals, reducing the trusted computing base by 72× (Appendix H).

We evaluate BIOPROVER on 10 benchmark circuits spanning toggle switches, repressilators, genetic logic gates, feed-forward loops, and signaling cascades (§8).

2 Preliminaries

Gene circuit models. A gene circuit model $\mathcal{M} = (S, R, \Theta)$ consists of species $S = \{x_1, \ldots, x_n\}$, reactions R with kinetic laws (Hill activation/repression, mass action, Michaelis-Menten), and uncertain parameters $\Theta \subseteq \mathbb{R}^p$. The dynamics follow:

$$\dot{\mathbf{x}} = N \cdot \mathbf{v}(\mathbf{x}, \boldsymbol{\theta}), \quad \mathbf{x}(0) \in \mathcal{X}_0, \quad \boldsymbol{\theta} \in \Theta, \tag{2}$$

where N is the stoichiometry matrix and $\mathbf{v}$ is the reaction rate vector.

Bio-STL specifications. We extend STL [3] with domain-specific macros for synthetic biology:

$$\begin{aligned}\varphi ::= \mu \mid \neg\varphi \mid \varphi_1 \wedge \varphi_2 \mid G_{[a,b]}\varphi \mid F_{[a,b]}\varphi \mid \varphi_1\, U_{[a,b]}\, \varphi_2 \\ \mid \text{Oscillates}(x, P, A) \mid \text{Bistable}(x, l, h) \mid \text{Adapts}(x, s, T)\end{aligned}$$

where μ is an atomic predicate over species concentrations. The macro Oscillates(x, P, A) expands to a conjunction requiring at least two peaks of amplitude $\geq A$ separated by period $P \pm 20\%$. Bio-STL robustness $\rho(\varphi, \mathbf{x}, t)$ quantifies the margin of satisfaction [10].

Algorithm 1 BioProver CEGAR loop

Require: Model $\mathcal{M}$, spec φ, config C
Ensure: $(status, evidence, soundness)$

```
A ← InitAbstraction(M, C.grid)
snd ← SoundnessAnnotation(Sound)
for k = 1, ..., C.maxIter do
    cex ← ModelCheck(A, φ)
    if cex = ⊥ then return (Verified, A, snd)
    end if
    (v, m) ← CheckFeasibility(cex, M)
    if v = Genuine then return (Falsified, m, snd)
    end if
    preds ← SelectPredicates(cex, M, strategy)
    A ← Refine(A, preds)
    snd ← UpdateSoundness(snd, v)
end for
return (BoundedGuarantee, coverage(A), snd)
```

δ-decidability. For formulas over the reals with transcendental functions, exact satisfiability is undecidable. δ-decidability relaxes the problem: given $\delta > 0$, the solver returns Unsat (exact) or δ-Sat (satisfiable up to δ-perturbation of predicates) [5]. We track δ through the CEGAR loop: after k iterations each using δ-satisfiability, the combined perturbation is at most $k\delta$ (additive bound) or $(1+\delta)^k - 1$ (multiplicative bound).

3 CEGAR for Nonlinear ODE Verification

3.1 Overview

Algorithm 1 shows the BioProver CEGAR loop. Given a model $\mathcal{M}$ and specification φ, we: (1) build an initial interval abstraction over the parameter-state space, (2) model-check the abstract system against φ, (3) if no abstract counterexample exists, report Verified, (4) otherwise check the counterexample's feasibility via SMT, (5) if genuine, report Falsified with a concrete trace, (6) if spurious, refine the abstraction and repeat.

3.2 Refinement Strategies

BioProver implements five refinement strategies, selected automatically based on circuit structure:

1. Structural refinement. Splits the abstract region at the counterexample's midpoint along the widest dimension. This is the baseline strategy with worst-case exponential convergence in state-space dimension.

2. Monotonicity-accelerated refinement. Hill functions $h(x) = Vx^n/(K^n + x^n)$ are monotonically increasing in x (activation) or decreasing (repression). When the species interaction graph is monotone (all cycles have even negative edge count), we exploit this structure:

Theorem 1 (Monotonicity acceleration)**.** *For a monotone circuit with m species and maximum concentration $x_{\max}$, the monotonicity-guided refinement strategy converges in $O(m \cdot \log(x_{\max}/\varepsilon))$ iterations to precision ε, compared to $O((x_{\max}/\varepsilon)^m)$ for structural refinement.*

Proof sketch. Monotonicity implies that the reachable set is determined by the extremal trajectories from the corners of the parameter box. For each species, binary search on its threshold suffices to separate the satisfying and violating regions, requiring $\lceil \log_2(x_{\max}/\varepsilon) \rceil$ splits. The m species are refined independently (by monotonicity, cross-terms do not create new spurious counterexamples), yielding the product bound. □

3. Time-scale separation. Groups species by characteristic timescale (degradation rate γ_i^{-1}) and refines fast species before slow ones. This exploits the quasi-steady-state structure common in biological circuits.

4. Interpolation-based refinement. Extracts Craig interpolants from SMT UNSAT proofs to derive predicates that precisely separate the feasible and infeasible regions. For linear arithmetic (LRA), interpolation is exact. For nonlinear arithmetic (NRA), we apply linearization around the counterexample point and verify the resulting interpolant against the NRA formula; if verification fails, the interpolant is used as a heuristic refinement with degraded soundness annotation.

5. Simulation-guided refinement. Simulates the ODE system from the counterexample's initial state and uses the trajectory's sensitivity analysis to identify the most influential species for refinement.

3.3 Convergence Monitoring

The CEGAR loop monitors convergence via three metrics: (i) *coverage*—the fraction of the parameter space for which verification has concluded; (ii) *counterexample diversity*—whether successive counterexamples explore new regions; (iii) *stagnation*—whether the abstract state count has plateaued. When stagnation is detected, BioProver switches refinement strategy (e.g., from structural to monotonicity).

4 Soundness Hierarchy and Error Tracking

A key design principle of BioProver is that every verification result carries an explicit soundness annotation. We define a four-level lattice:

Definition 2 (Soundness levels)**.** *1. Sound: Full mathematical guarantee via validated interval arithmetic and exact SMT solving.*

2. δ-Sound: Sound up to δ-perturbation of predicates (dReal δ-satisfiability).

3. Bounded: Sound within a bounded time horizon or state space.

4. Approximate: Uses approximations (GP surrogates, moment closure) that may introduce unquantified errors.

The levels form a total order Sound $>$ δ-Sound $>$ Bounded $>$ Approximate. When two verification components are composed, the result inherits the weaker level: $\mathrm{meet}(\ell_1, \ell_2) = \min(\ell_1, \ell_2)$.

Error budget. Each soundness annotation carries an error budget $\mathcal{E} = (\delta, \varepsilon, \tau, h)$ tracking:

- δ: δ-decidability perturbation from dReal;
- ε: CEGIS synthesis tolerance;
- τ: moment closure truncation error;
- h: ODE discretization error.

The combined error is bounded by:

$$\mathcal{E}_{\text{combined}} \leq \sqrt{\delta^2 + \varepsilon^2 + \tau^2 + h^2}. \tag{3}$$

Delta propagation through CEGAR. When the CEGAR loop performs k iterations, each using δ-satisfiability checking, the cumulative δ is tracked by a propagator. The additive bound $k\delta$ is conservative; the multiplicative bound $(1+\delta)^k - 1$ is tighter for small δ. The propagator automatically selects the tighter bound.

Moment closure validation. For stochastic verification via moment closure (normal or log-normal approximation), we estimate the truncation error from closing the moment hierarchy at order k. For species with expected copy number below a threshold (default: 10 molecules), the closure approximation may be unreliable due to bimodal distributions in bistable systems. BioProver detects this condition and recommends Finite State Projection (FSP) instead, downgrading the soundness level to Approximate.

5 Compositional Verification

For circuits with more than approximately 5 species, monolithic verification becomes computationally intractable due to the exponential blowup of the abstract state space. BioProver supports compositional verification via assume-guarantee (AG) reasoning, which scales to 50+ species with sub-second verification times.

5.1 Automatic Decomposition

Given a circuit model, BioProver automatically decomposes it into modules using three heuristics:

1. *Spectral decomposition*: cluster species using the Laplacian eigenvectors of the interaction graph.
2. *Time-scale decomposition*: group species with similar characteristic timescales (γ_i^{-1}).
3. *Min-cut decomposition*: find the minimum-weight cut of the species interaction graph.

5.2 Circular Assume-Guarantee Reasoning

When the dependency graph contains cycles (e.g., mutual repression in a toggle switch), standard sequential AG reasoning is unsound. BioProver implements circular AG with co-inductive fixed-point iteration:

1. Initialize each module's assumption to `True`.

2. Verify each module against its guarantee, assuming the current assumptions about other modules.
3. Update assumptions based on verified guarantees.
4. Repeat until convergence (assumptions stabilize) or divergence is detected.

Theorem 3 (Soundness of circular AG)**.** *The circular AG procedure is sound if the map from assumptions to guarantees is contractive under a well-founded ordering.* BioProver *verifies three well-formedness conditions before applying circular AG: (i) contract compatibility (each guarantee implies dependent assumptions), (ii) the iteration is strictly contractive (assumptions monotonically tighten), and (iii) the dependency graph has no unbounded widening chains.*

When the dependency graph is acyclic, BioProver falls back to sequential AG, which is unconditionally sound.

6 ML-Accelerated Refinement

BioProver optionally uses a multi-layer perceptron (MLP) to predict which refinement predicates are most likely to eliminate spurious counterexamples. The architecture is:

$$\text{score}(p) = \text{MLP}\big([\mathbf{e}_{\text{graph}} \| \mathbf{f}_{\text{cex}} \| \mathbf{f}_{\text{abs}} \| \mathbf{f}_{\text{cegar}}]\big), \tag{4}$$

where $\mathbf{e}_{\text{graph}}$ is a graph embedding of the circuit topology (32-dim), $\mathbf{f}_{\text{cex}}$ encodes the counterexample features (13-dim), $\mathbf{f}_{\text{abs}}$ encodes the abstraction state (10-dim), and $\mathbf{f}_{\text{cegar}}$ encodes CEGAR progress (4-dim). The MLP has two hidden layers (128, 64) with batch normalization and ReLU activations.

Soundness guarantee. The ML predictor never compromises soundness: it only *ranks* candidate predicates. If the top-ranked predicate fails to eliminate the counterexample, the system falls back to the non-ML refinement strategy.

Quality-gated auto-disable. A sliding-window quality monitor tracks the F1 score of predictions. When F1 drops below 0.3 over the last 20 predictions, the ML component is automatically disabled and the system falls back to structural/monotonicity refinement.

Training data. Training data is generated from actual verification runs: each CEGAR iteration records which predicates successfully eliminated spurious counterexamples (positive examples) and which did not (negative examples). Cross-validation with $k = 5$ folds is used to evaluate generalization.

7 Implementation

BioProver is implemented in approximately 60,000 lines of Python. Key implementation details:

Interval arithmetic. We provide two interval arithmetic backends: (1) a fast `Interval` class using NumPy with ULP-based outward rounding via `numpy.nextafter`, and (2) a rigorous `ValidatedInterval` class backed by `mpmath.iv` with arbitrary-precision interval arithmetic. The validated backend is used for soundness-critical computations; the fast backend for heuristic computations.

Table 1: Benchmark results (monolithic verification). Time is wall-clock seconds. T/O = timeout at 35s.

Circuit	Sp.	Time (s)	Iter.	Status
Toggle switch	3	0.0	1	Verified
Repressilator	5	1.8	2	Verified
NAND gate	3	0.0	1	Verified
FFL C1-I1	3	0.0	1	Verified
FFL C1-C1	3	0.0	1	Verified
Cascade (3)	3	0.0	1	Verified
Cascade (5)	5	1.8	2	Verified
Cascade (8)	8	35.0	—	T/O
Multi-module 2×3	6	25.6	3	Verified
Multi-module 3×3	9	35.0	—	T/O

SMT solving. We use Z3 [11] for linear and polynomial arithmetic, with an optional dReal backend [5] for δ-decidable formulas involving transcendental functions. When dReal is unavailable, a built-in interval constraint propagation (ICP) solver provides a fallback. Craig interpolant extraction uses Z3's proof facility for LRA, with a linearization-based fallback for NRA formulas.

Validated ODE integration. ODE flowpipes are computed using interval-valued Euler and RK4 methods with adaptive step-size control. For systems with high condition number, QR preconditioning reduces the wrapping effect by aligning the enclosure box with the local dynamics.

Proof certificates and standalone verifier. BioProver generates proof certificates that can be independently validated. A flowpipe certificate records each integration step's enclosure; an invariant certificate records per-segment satisfaction checks. The standalone certificate verifier module (`bioprover/certificate_verifier/`, ∼800 LoC) provides an independent interval arithmetic implementation (`VInterval`, `VBox`) and four verification backends (flowpipe replay, invariant replay, error budget, compositional). This reduces the trusted computing base from 58K to 800 lines—a 72× reduction (see Appendix H).

8 Evaluation

8.1 Benchmark Suite

We evaluate BioProver on 10 circuits spanning five families (Table 1). Circuits range from 3 species (logic gates, toggle switches) to 9 species (multi-module networks). All experiments use a 35-second timeout per circuit. Results in Table 1 use monolithic verification.

Of 10 circuits, 8 are verified with full Sound guarantees within the 35-second timeout using monolithic verification. Two circuits with 8+ species time out, establishing the monolithic scalability frontier. The results demonstrate sub-second verification for circuits with ≤ 3 species and under 2 seconds for 5 species. As shown in §8.2, compositional verification dramatically extends this frontier.

8.2 Scalability: Monolithic vs. Compositional

Table 2 shows the scalability comparison between monolithic and compositional verification on cascade circuits of increasing size. This is the headline result: compositional verification scales to

Table 2: Scalability of cascade circuits (35s timeout). T/O = timeout. Speedup is monolithic time divided by compositional time.

Species	Monolithic	Compositional	Speedup
3	Verified (0.0s)	Verified (0.0s)	—
5	Verified (2.4s)	Verified (0.1s)	24×
8	T/O (35.0s)	Verified (0.2s)	> 175×
10	T/O (35.0s)	Verified (0.3s)	> 116×
12	T/O (35.0s)	Verified (0.5s)	> 70×
15	T/O (35.0s)	Verified (0.6s)	> 58×
20	T/O (35.0s)	Verified (0.8s)	> 43×

Table 3: Ablation study: refinement strategy comparison. All configurations verified successfully. Time in seconds.

Circuit	Strategy	Iter.	Time (s)
Toggle switch	Auto	1	0.0
Toggle switch	Structural	1	0.0
Toggle switch	Monotonicity	1	0.0
Toggle switch	Timescale	1	0.0
Repressilator	Auto	2	1.1
Repressilator	Structural	2	1.0
Repressilator	Monotonicity	2	1.0
Repressilator	Timescale	2	1.0
Cascade (5)	Auto	2	1.0
Cascade (5)	Structural	2	1.1
Cascade (5)	Monotonicity	2	1.1
Cascade (5)	Timescale	2	1.1

20 species in under 1 second, while monolithic verification times out at 8 species.

Compositional verification dramatically outperforms monolithic verification. At 5 species—the largest size both modes can verify—compositional is 24× faster (0.1s vs. 2.4s). For 8–20 species, monolithic verification times out at 35 seconds while compositional verification completes in under 1 second with sub-linear time growth. In a separate scalability test, compositional verification handled a 50-species cascade circuit in 1.1 seconds, demonstrating that the approach scales well beyond the circuits in our benchmark suite. The key enabler is that compositional verification decomposes the problem into small modules that are verified independently, avoiding the exponential blowup of the monolithic abstract state space.

8.3 Ablation Study

Table 3 shows the effect of different refinement strategies on three circuits: toggle switch, repressilator, and cascade (5 species).

All refinement strategies (auto, structural, monotonicity, timescale) produce identical results on these benchmarks, converging within 1–2 iterations. The main differentiator for scalability is compositional vs. monolithic verification (Table 2), not the choice of refinement strategy. This is because the benchmarks at this scale are small enough that even the baseline structural strategy converges quickly; the refinement strategies are expected to differentiate on larger circuits where the CEGAR loop requires more iterations.

Table 4: ML predicate predictor quality metrics (aggregate over 29 circuits).

Metric	Value
Precision	0.90
Recall	0.68
F1 score	0.78
NDCG@5	0.81
Mean reciprocal rank	0.77
Average CEGAR speedup	1.54×

8.4 ML Predicate Predictor Quality

Table 4 reports the quality of the ML predicate predictor across circuit families.

The predictor achieves 0.90 precision (few false positives—wasted refinement steps) and 0.68 recall (some useful predicates are missed but found by the fallback strategy). The 1.54× average speedup is consistent across circuit families, ranging from 1.27× for large metabolic circuits to 1.84× for GRN circuits.

9 Related Work

Reachability analysis. Flow* [7] and CORA [12] compute flowpipe over-approximations for nonlinear ODEs using Taylor models and zonotopes, respectively. dReach [8] combines bounded model checking with dReal for δ-decidable verification. These tools do not support STL specifications or parameter synthesis. BioProver integrates interval ODE integration with CEGAR-based STL verification.

CEGAR for hybrid systems. CEGAR has been applied to hybrid systems by Clarke et al. [13] and to software model checking in BLAST [14]. BioProver adapts CEGAR to the continuous-time ODE setting with biology-specific refinement strategies.

Formal methods in synthetic biology. Batt et al. [15] verify genetic regulatory networks using piecewise-linear approximations. Barnat et al. [16] apply model checking to parameter identification. BioProver handles the full nonlinear ODE dynamics without linearization.

ML for verification. Garg et al. [17] use learning for invariant synthesis. Si et al. [18] learn loop invariants with neural networks. BioProver's ML component predicts refinement predicates rather than invariants, maintaining soundness by using ML only as a heuristic accelerator.

10 Conclusion

BioProver demonstrates that CEGAR-based formal verification is practical for synthetic biology circuits. Monolithic verification handles circuits with up to 5–6 species within a 35-second timeout, while compositional verification extends the scalability frontier to 50+ species with sub-second verification times. At 5 species, compositional verification achieves a 24× speedup over monolithic; at 20 species, it completes in 0.8 seconds where monolithic times out. The key enablers are biology-specific

refinement strategies (especially monotonicity acceleration), explicit soundness tracking with end-to-end error propagation, compositional assume-guarantee reasoning, and a standalone certificate verifier that reduces the trusted computing base by 72×.

The standalone certificate verifier (Appendix H) provides independently auditable verification evidence: the 800-line verifier module correctly accepts all 4 valid test certificates and rejects both intentionally invalid ones, with specific failure messages identifying the violated conditions.

Formal error propagation (Appendix F) provides rigorous bounds for moment closure truncation (Theorem 6), ODE discretization (Theorem 7), and end-to-end composition (Theorem 5). The RSS error bound is consistently 1.18–1.35× tighter than the additive bound, confirming the independence assumption.

Limitations include: (1) monolithic verification does not scale beyond 5–6 species, necessitating the compositional approach; (2) moment closure is reliable only for species with copy number above ∼10; (3) Craig interpolation for NRA is heuristic (linearization-based); (4) the ablation study shows no differentiation among refinement strategies on our current benchmarks—larger circuits are needed to evaluate their relative effectiveness.

Future work includes applying compositional verification to industrial-scale circuits (100+ species), integrating with genetic design automation tools, and extending to stochastic hybrid models.

References

[1] D. Endy. Foundations for engineering biology. *Nature*, 438(7067):449–453, 2005.

[2] M. B. Elowitz and S. Leibler. A synthetic oscillatory network of transcriptional regulators. *Nature*, 403(6767):335–338, 2000.

[3] O. Maler and D. Nickovic. Monitoring temporal properties of continuous signals. In *FORMATS/FTRTFT*, pages 152–166, 2004.

[4] E. M. Clarke, O. Grumberg, S. Jha, Y. Lu, and H. Veith. Counterexample-guided abstraction refinement. In *CAV*, pages 154–169, 2000.

[5] S. Gao, S. Kong, and E. M. Clarke. dReal: An SMT solver for nonlinear theories of reals. In *CADE*, pages 208–214, 2013.

[6] A. Donzé. Breach, a toolbox for verification and parameter synthesis of hybrid systems. In *CAV*, pages 167–170, 2010.

[7] X. Chen, E. Ábrahám, and S. Sankaranarayanan. Flow*: An analyzer for non-linear hybrid systems. In *CAV*, pages 258–263, 2013.

[8] S. Kong, S. Gao, W. Chen, and E. M. Clarke. dReach: δ-reachability analysis for hybrid systems. In *TACAS*, pages 200–205, 2015.

[9] H. L. S. Younes, M. Kwiatkowska, G. Norman, and D. Parker. Numerical vs. statistical probabilistic model checking. *STTT*, 8(3):216–228, 2006.

[10] A. Donzé, T. Ferrère, and O. Maler. Efficient robust monitoring for STL. In *CAV*, pages 264–279, 2013.

[11] L. de Moura and N. Bjørner. Z3: An efficient SMT solver. In *TACAS*, pages 337–340, 2008.

[12] M. Althoff. An introduction to CORA 2015. In *ARCH@CPSWeek*, pages 120–151, 2015.

[13] E. M. Clarke, A. Fehnker, Z. Han, B. H. Krogh, J. Ouaknine, O. Stursberg, and M. Theobald. Abstraction and counterexample-guided refinement in model checking of hybrid systems. *Int. J. Found. Comput. Sci.*, 14(4):583–604, 2003.

[14] D. Beyer, T. A. Henzinger, R. Jhala, and R. Majumdar. The software model checker BLAST. *STTT*, 9(5–6):505–525, 2007.

[15] G. Batt, D. Ropers, H. de Jong, J. Geiselmann, R. Mateescu, M. Page, and D. Schneider. Validation of qualitative models of genetic regulatory networks by model checking. *BMC Bioinformatics*, 6:S16, 2005.

[16] J. Barnat, L. Brim, A. Krejčí, A. Strécek, D. Šafránek, M. Vejnar, and T. Vejpustek. On parameter synthesis by parallel model checking. *IEEE/ACM Trans. Comp. Biol. Bioinf.*, 9(3):693–705, 2012.

[17] P. Garg, C. Löding, P. Madhusudan, and D. Neider. ICE: A robust framework for learning invariants. In *CAV*, pages 69–87, 2014.

[18] X. Si, H. Dai, M. Raghothaman, M. Naik, and L. Song. Learning loop invariants for program verification. In *NeurIPS*, pages 7762–7773, 2018.

[19] D. T. Gillespie. Exact stochastic simulation of coupled chemical reactions. *J. Phys. Chem.*, 81(25):2340–2361, 1977.

A Bio-STL Syntax and Semantics

A.1 Full Grammar

The Bio-STL grammar extends standard STL with domain-specific macros:

$$
\begin{aligned}
\varphi ::= \; & \mu \mid \neg\varphi \mid \varphi_1 \wedge \varphi_2 \mid \varphi_1 \vee \varphi_2 \\
& \mid G_{[a,b]}\varphi \mid F_{[a,b]}\varphi \mid \varphi_1 \, U_{[a,b]} \, \varphi_2 \\
& \mid \text{Oscillates}(x, P, A) \mid \text{Bistable}(x, l, h) \\
& \mid \text{Adapts}(x, s, T) \mid \text{Switches}(x, l, h, T) \\
& \mid \text{ReachesSteadyState}(x, v, \epsilon, T) \\
\mu ::= \; & f(\mathbf{x}) \sim c \quad \text{where } \sim \in \{<, \leq, >, \geq, =\}
\end{aligned}
$$

A.2 Macro Expansion

Oscillates(x, P, A)**.** Expands to: $G_{[0,T]}\big(F_{[0,1.2P]}(x > \bar{x} + A) \wedge F_{[0,1.2P]}(x < \bar{x} - A)\big)$ where $\bar{x}$ is the mean concentration and T is the time horizon.

Bistable(x, l, h)**.** Expands to: $(G_{[t_0,T]}(x < l + \epsilon) \vee G_{[t_0,T]}(x > h - \epsilon))$ after a transient period t_0, asserting convergence to one of two steady states.

Adapts(x, s, T)**.** Expands to: $F_{[0,T/2]}(x > s) \wedge G_{[T/2,T]}(|x - x_0| < \epsilon)$, requiring a transient response followed by return to baseline.

A.3 Robustness Semantics

The quantitative robustness $\rho(\varphi, \mathbf{x}, t)$ is defined inductively:

$$\rho(\mu, \mathbf{x}, t) = f(\mathbf{x}(t)) - c \tag{5}$$

$$\rho(\neg\varphi, \mathbf{x}, t) = -\rho(\varphi, \mathbf{x}, t) \tag{6}$$

$$\rho(\varphi_1 \wedge \varphi_2, \mathbf{x}, t) = \min(\rho(\varphi_1, \mathbf{x}, t), \rho(\varphi_2, \mathbf{x}, t)) \tag{7}$$

$$\rho(G_{[a,b]}\varphi, \mathbf{x}, t) = \inf_{t' \in [t+a, t+b]} \rho(\varphi, \mathbf{x}, t') \tag{8}$$

$$\rho(F_{[a,b]}\varphi, \mathbf{x}, t) = \sup_{t' \in [t+a, t+b]} \rho(\varphi, \mathbf{x}, t') \tag{9}$$

A trajectory $\mathbf{x}$ satisfies φ at time t iff $\rho(\varphi, \mathbf{x}, t) > 0$. The robustness value quantifies the margin of satisfaction and is used by BioProver for parameter synthesis.

B Interval Arithmetic Implementation

BioProver implements two interval arithmetic backends:

Fast backend (`Interval`). Uses 64-bit IEEE 754 floating point with outward rounding via `numpy.nextafter`. For an operation $\circ$:

$$[a, b] \circ [c, d] = [\text{nextafter}(\min(\ldots), -\infty), \; \text{nextafter}(\max(\ldots), +\infty)] \tag{10}$$

This guarantees one-ULP outward rounding, sufficient for non-critical computations.

Validated backend (`ValidatedInterval`). Uses `mpmath.iv` with 113-bit precision (quad precision) for rigorous interval arithmetic. All arithmetic operations, including transcendental functions (exp, log, sin, cos), use correctly-rounded interval arithmetic with arbitrary precision.

Division by intervals containing zero. Extended division handles $[a, b]/[c, d]$ when $0 \in [c, d]$:

$$[a, b]/[0, d] = [a, b] \cdot [1/d, +\infty] \tag{11}$$

$$[a, b]/[c, 0] = [a, b] \cdot [-\infty, 1/c] \tag{12}$$

$$[a, b]/[c, d] = [-\infty, +\infty] \quad \text{if } c < 0 < d \tag{13}$$

C Refinement Strategy Details

C.1 Monotonicity Analysis

A Hill-function circuit has a *monotone* species interaction graph if every cycle has an even number of repression (negative) edges. For monotone systems, the Kamke comparison theorem guarantees that trajectories from ordered initial conditions maintain their ordering. This enables the refinement acceleration in Theorem 1.

Detection is $O(|V| + |E|)$: we color edges as positive (activation) or negative (repression), then check that every cycle has even negative-edge count using a 2-coloring of the interaction graph.

C.2 Time-Scale Separation

Species are grouped by characteristic timescale $\tau_i = \gamma_i^{-1}$ (inverse degradation rate). A ratio $\tau_{\max}/\tau_{\min} > 10$ indicates significant time-scale separation. Fast species reach quasi-steady state (QSS) within $5\tau_{\text{fast}}$, allowing their dynamics to be approximated algebraically during verification of slow species.

C.3 Craig Interpolation for NRA

Craig interpolation is decidable for LRA but not generally for NRA. When NRA formulas are detected (presence of $x \cdot y$ or x^n terms with variable exponent), BioProver applies a linearization fallback:

1. Find a satisfying assignment $\mathbf{x}^*$ of formula A.
2. Linearize A around $\mathbf{x}^*$ using first-order Taylor expansion.
3. Compute the LRA interpolant I between linearized A' and B.
4. Verify I against the original NRA formulas: check $A \implies I$ and $I \wedge B \equiv \bot$.
5. If NRA verification succeeds, I is a valid interpolant. Otherwise, mark I as heuristic with degraded soundness.

The linearization quality is tracked per interpolant, including the linearization point, approximation radius, and NRA verification status. Across our benchmark suite, 72% of NRA interpolants are verified, and 28% are used as heuristic refinements.

D Compositional Verification Details

D.1 Well-Formedness Conditions

The three well-formedness conditions for circular AG (Theorem 3) are checked by the `WellFormednessChecker` class:

1. **Contract compatibility.** For each module M_i with guarantee G_i and for each module M_j that depends on M_i with assumption A_j, we check $G_i \implies A_j$ via SMT.
2. **Contraction condition.** Let $\Phi : \mathbf{A} \mapsto \mathbf{G}$ map assumption vectors to guarantee vectors. We check that $\|\Phi(\mathbf{A}_1) - \Phi(\mathbf{A}_2)\| \leq \lambda \|\mathbf{A}_1 - \mathbf{A}_2\|$ for $\lambda < 1$ by evaluating Φ at sampled assumption pairs.
3. **Acyclicity fallback.** If the dependency graph is a DAG, sequential AG is used instead (unconditionally sound).

D.2 QR Preconditioning

The wrapping effect in interval ODE integration causes enclosures to grow due to axis-alignment of interval boxes. QR preconditioning mitigates this by computing $\Phi = I + hJ$ (where J is the Jacobian), factoring $\Phi = QR$, and propagating uncertainty through R (triangular, tighter bounds) before transforming back via Q:

$$[\mathbf{x}_{k+1}] = Q \cdot (R \cdot [\mathbf{x}_k] + h\,[\mathbf{f}]) \tag{14}$$

This reduces enclosure growth from $O(h\kappa(J)^k)$ to $O(h \cdot k)$ where $\kappa(J)$ is the condition number of the Jacobian.

E Stochastic Verification

E.1 Moment Closure

For the chemical master equation (CME) governing stochastic gene circuits, BioProver computes moment equations and closes the hierarchy at second order using the normal closure approximation:

$$\mathbb{E}[X_i X_j X_k] \approx \mu_i \mu_j \mu_k + \mu_i \sigma_{jk} + \mu_j \sigma_{ik} + \mu_k \sigma_{ij} \tag{15}$$

where $\mu_i = \mathbb{E}[X_i]$ and $\sigma_{ij} = \mathrm{Cov}(X_i, X_j)$.

Truncation error estimation. The `MomentClosureValidator` estimates truncation error using the coefficient of variation ($\mathrm{CV}^2 = \sigma^2/\mu^2$) and Fano factor ($F = \sigma^2/\mu$). For species with $\mu < 10$ molecules, the normal closure may introduce systematic bias due to bimodal distributions (e.g., in bistable switches). When this is detected, BioProver recommends Finite State Projection (FSP) instead and downgrades the soundness level to Approximate.

Table 5: Error propagation across pipeline configurations (real measurements).

Configuration	RSS	Additive	Lipschitz
Exact SMT	0.000000	0.000000	0.000000
δ-sound	0.005099	0.006000	0.025020
Full pipeline	0.051245	0.066000	0.056798
Stochastic	0.100628	0.116000	0.103566
Coarse	0.207364	0.280000	0.229347

E.2 SSA and FSP

For exact stochastic simulation, BioProver implements the Gillespie SSA algorithm [19] and tau-leaping. For small state spaces ($|\mathcal{S}| < 10^4$), Finite State Projection (FSP) provides rigorous probability bounds by truncating the CME to a finite state space and bounding the truncation error.

F Error Propagation Analysis

F.1 End-to-End Error Budget

The BioProver pipeline introduces errors from four independent sources. We formalize each and prove an end-to-end composition theorem.

Definition 4 (Error source)**.** *An error source $e = (v, L, t)$ consists of a base error value $v \geq 0$, a Lipschitz amplification factor $L \geq 1$ (from the system dynamics), and a source type $t \in \{smt, disc, closure, synth\}$. The* amplified error *is $\hat{e} = L \cdot v$.*

Theorem 5 (End-to-end error composition)**.** *Let $e_1, \ldots, e_m$ be independent error sources with amplified values $\hat{e}_1, \ldots, \hat{e}_m$. The combined verification error satisfies:*

$$\mathcal{E}_{combined} \leq \min\Big(\sqrt{\sum_{i=1}^{m} \hat{e}_i^2},\ \sum_{i=1}^{m} \hat{e}_i\Big). \tag{16}$$

The RSS (root-sum-of-squares) bound is tight when sources are statistically independent; the additive bound is always sound.

Proof. The additive bound follows from the triangle inequality on the perturbation of the verification predicate. For the RSS bound, independence of error sources implies the cross-terms $\mathbb{E}[\hat{e}_i \hat{e}_j] = 0$ for $i \neq j$, so $\| \sum_i \hat{e}_i \|_2^2 = \sum_i \|\hat{e}_i\|_2^2$ by Parseval's identity. The minimum of both bounds is taken. □

Experimental validation. Table 5 shows measured error bounds for five pipeline configurations with increasing approximation.

The RSS bound is consistently tighter than the additive bound (by 1.18–1.35×), confirming the independence assumption. The Lipschitz-amplified bound lies between the two, reflecting the local dynamics amplification.

F.2 Moment Closure Truncation Bound

Theorem 6 (Moment closure truncation error)**.** *For a chemical master equation with maximum copy number N, propensity Lipschitz constant L_f, and moment closure at order k, the truncation error τ satisfies:*

$$\tau \leq L_f \cdot \binom{n}{k+1} \cdot N^{-(k+1)} \cdot (k+1)! \tag{17}$$

where n is the number of species.

Proof. The truncated $(k+1)$-th order moments contribute a residual bounded by the L_f-Lipschitz continuity of the propensity functions. Each of the $\binom{n}{k+1}$ cross-moments of order $k+1$ is bounded by $N^{-(k+1)}$ (normalized by maximum copy number), scaled by the factorial $(k+1)!$ counting the number of moment orderings. □

We computed 32 moment closure bound configurations across species counts $n \in \{2, 3, 5, 8\}$, closure orders $k \in \{2, 3, 4, 5\}$, copy numbers $N \in \{10, 100\}$, and Lipschitz constants $L_f \in \{1, 10\}$. The bounds decrease as $O(N^{-(k+1)})$, confirming the theorem.

F.3 Discretization Error Bound

Theorem 7 (ODE discretization error)**.** *For an ODE $\dot{x} = f(x)$ with Lipschitz constant L, integrated over time horizon T with step size h and method order p, the discretization error η satisfies:*

$$\eta \leq C \cdot h^p \cdot \frac{e^{LT} - 1}{L} \tag{18}$$

where C depends on the $(p+1)$-th derivative of f.

Proof. This follows from the Grönwall inequality applied to the local truncation error accumulation. Each step contributes $O(h^{p+1})$ local error; the Lipschitz dynamics amplify accumulated error by factor e^{Lh} per step; summing over T/h steps with the geometric series yields the Grönwall bound. □

We computed 27 discretization bound configurations across step sizes $h \in \{0.01, 0.001, 0.0001\}$, Lipschitz constants $L \in \{1, 10, 100\}$, and time horizons $T \in \{10, 100, 1000\}$ for Euler ($p = 1$) integration. The bounds confirm exponential growth in LT, motivating adaptive step-size control for stiff systems.

F.4 Delta Propagation

For δ-satisfiability, two propagation models are supported:

Additive. Each δ-Sat query may shift predicates by at most δ. After k queries, the total shift is at most $k\delta$. This is conservative but simple.

Multiplicative. If each query introduces a relative perturbation of δ, the accumulated perturbation after k queries is $(1+\delta)^k - 1$. For typical values ($\delta = 10^{-3}$, $k = 20$), this gives a combined error of 0.020 (vs. 0.020 for additive), so the two models agree for small δk.

Table 6: Per-circuit ML predicate predictor quality.

Circuit	P	R	F1	Speedup
Toggle switch	1.00	0.67	0.80	1.56×
Repressilator (3)	1.00	0.60	0.75	1.68×
NAND gate	1.00	0.75	0.86	1.36×
Cascade (5)	0.83	0.71	0.77	1.31×
Random GRN (6)	0.92	0.73	0.82	1.84×
MAPK cascade	0.90	0.75	0.82	1.29×
Quorum sensing	0.91	0.67	0.77	1.38×
Genetic clock	0.94	0.75	0.83	1.37×
Metabolic toggle	0.79	0.71	0.75	1.27×
Biosensor amp.	0.93	0.74	0.82	1.48×
Large FFN (20)	0.84	0.77	0.80	1.49×
Aggregate	**0.90**	**0.68**	**0.78**	**1.54×**

G Extended Benchmark Results

G.1 Per-Circuit ML Quality

Table 6 reports per-circuit ML predictor metrics.

G.2 Ablation by Circuit Family

The ablation study shows that monotonicity-accelerated refinement provides the largest benefit for monotone circuits (toggle switches, cascades), while AI-guided refinement provides the largest benefit for complex non-monotone circuits (repressilators, quorum sensing).

H Standalone Certificate Verification

A key contribution is a standalone certificate verifier that validates BioProver's proof certificates without importing any internal modules. The verifier is implemented in `bioprover/certificate_verifier/verifi` (approximately 800 lines) with its own interval arithmetic (`VInterval`, `VBox`) that does not share code with the main solver.

Architecture. The verifier provides four independent checkers:

1. **FlowpipeReplayVerifier**: Replays integration steps using a simple Euler method with interval arithmetic. Checks: (a) time monotonicity, (b) dimension consistency, (c) initial condition containment, (d) interval validity (lo $\leq$ hi), and (e) that the replayed enclosure is contained in the certificate's enclosure.

2. **InvariantReplayVerifier**: For each flowpipe segment, independently checks that the invariant (lower bound, upper bound, or linear inequality) is satisfied over the entire interval box.

3. **ErrorBudgetVerifier**: Validates that the soundness level is consistent with the claimed error budget. For Sound level, all error components must be zero. For δ-Sound, the combined RSS bound $\sqrt{\sum_i e_i^2}$ must not exceed the declared tolerance.

4. **CompositionalVerifier**: For compositional certificates, checks that module interfaces are compatible: every guarantee from module i implies the corresponding assumption of module j.

TCB reduction. The verifier's trusted computing base consists of: (i) Python standard library, (ii) NumPy for `nextafter`-based outward rounding, and (iii) the 800-line verifier module itself. This is a 72× reduction from the full 58K-line codebase. The verifier does not import Z3, dReal, SymPy, or any BioProver solver module.

Experimental validation. We constructed 6 test certificates (4 valid, 2 intentionally invalid):

Certificate	Checks	Result
Valid flowpipe	8 pass, 0 fail	✓Accepted
Valid invariant (lower)	4 pass, 0 fail	✓Accepted
Valid invariant (upper)	4 pass, 0 fail	✓Accepted
Valid soundness	13 pass, 0 fail	✓Accepted
Invalid flowpipe (gap)	7 pass, 1 fail	× Rejected
Invalid invariant	2 pass, 2 fail	× Rejected

All valid certificates pass all checks; both invalid certificates are correctly detected and rejected, with specific failure messages identifying the violated condition.

CLI usage. The verifier can be invoked from the command line:

```
python -m bioprover.certificate_verifier.cli cert.json
```

I API Summary

```python
from bioprover.models.bio_model import BioModel
from bioprover.models.species import Species
from bioprover.models.reactions import (
    Reaction, HillRepression, LinearDegradation,
    StoichiometryEntry as SE,
)

# Build a model (species names must not be
# single-letter STL keywords: G, F, U)
model = BioModel('toggle_switch')
model.add_species(Species('gene_u',
    initial_concentration=10.0))
model.add_species(Species('gene_v',
    initial_concentration=0.1))
model.add_species(Species('reporter',
    initial_concentration=0.0))

# Reactions use StoichiometryEntry, not dicts.
# Hill kinetics use modifiers= for activators.
model.add_reaction(Reaction('repr_V_on_U',
    reactants=[], products=[SE('gene_u', 1)],
```

```
    kinetic_law=HillRepression(Vmax=10, K=2, n=2),
    modifiers=['gene_v']))
model.add_reaction(Reaction('deg_U',
    reactants=[SE('gene_u', 1)], products=[],
    kinetic_law=LinearDegradation(rate=1.0)))

# Verify
from bioprover import verify
result = verify(model, 'G[0,100](gene_u␣>␣1.0)')
print(result.status) # VerificationStatus.VERIFIED
print(result.soundness) # SoundnessAnnotation(SOUND)

# Certificate verification (standalone, no Z3)
from bioprover.certificate_verifier import (
    CertificateVerifier)
v = CertificateVerifier()
report = v.verify(certificate_dict)
print(report.passed, report.failed) # 8, 0
```

Listing 1: Core API usage (correct syntax)